

BCT Lab Softcopies

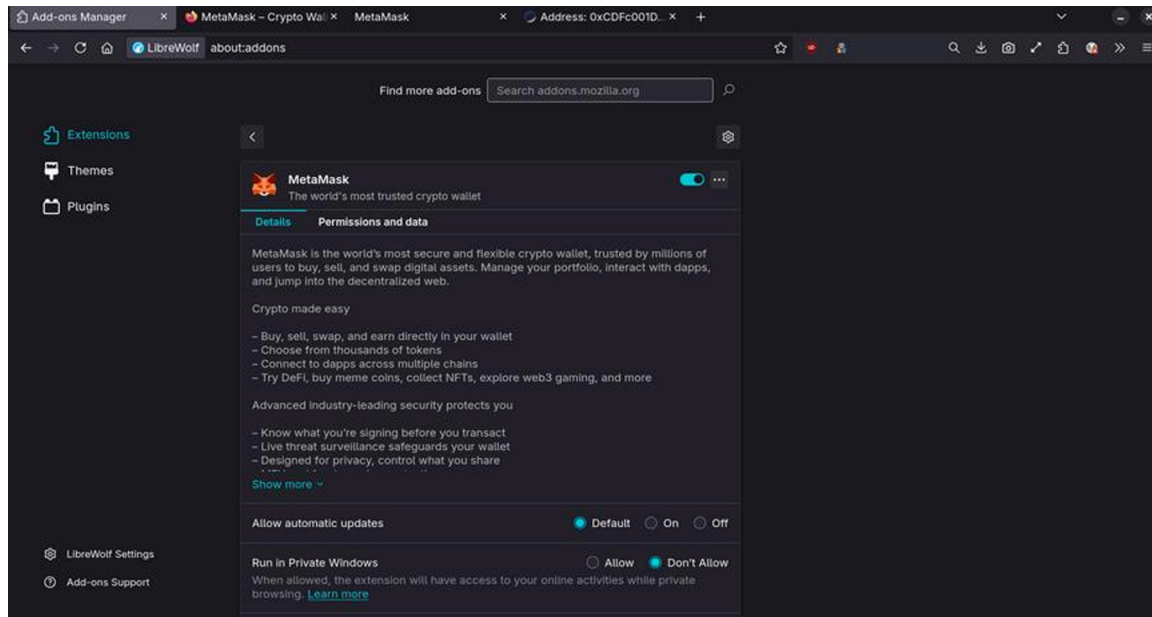
Name: Priya Rathod

Roll No.: 45

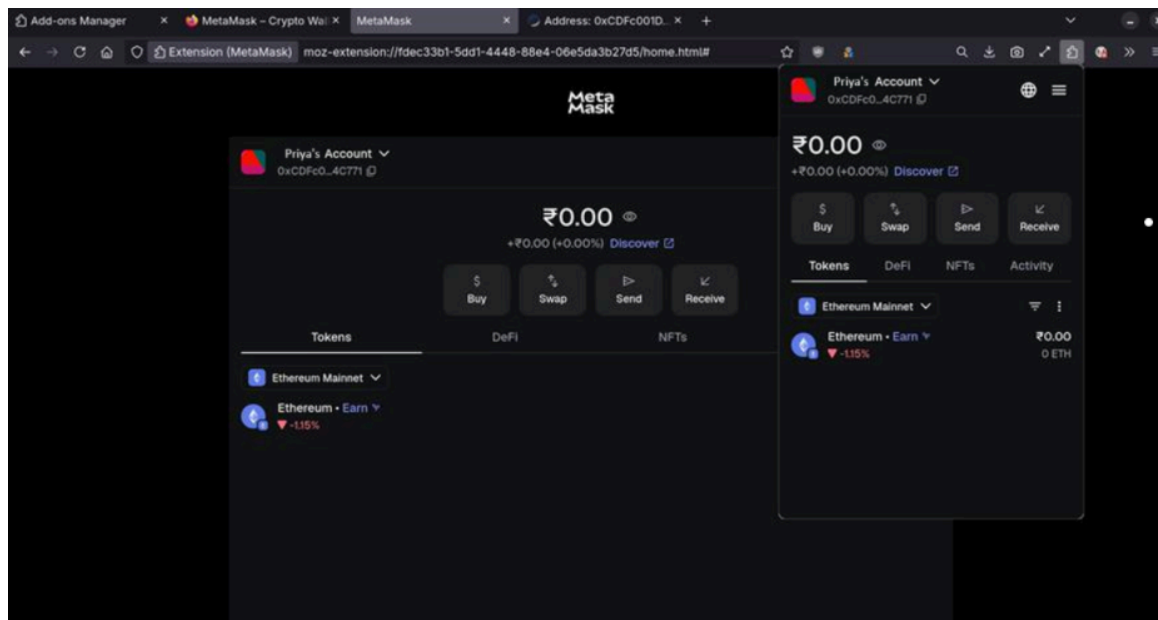
PRN: F22111052

Class: B.E. COMP 1 - Q Batch

Practical 1 (Installation of Metamask)



Practical 2 (Creating Wallet)



Practical 3 (Bank Account)

Code:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract BankAccount {
    mapping(address => uint256) private balances;

    event Deposit(address indexed account, uint256 amount);
    event Withdraw(address indexed account, uint256 amount);

    function deposit() public payable {
        require(msg.value > 0, "Deposit amount must be greater than 0");
        balances[msg.sender] += msg.value;
        emit Deposit(msg.sender, msg.value);
    }

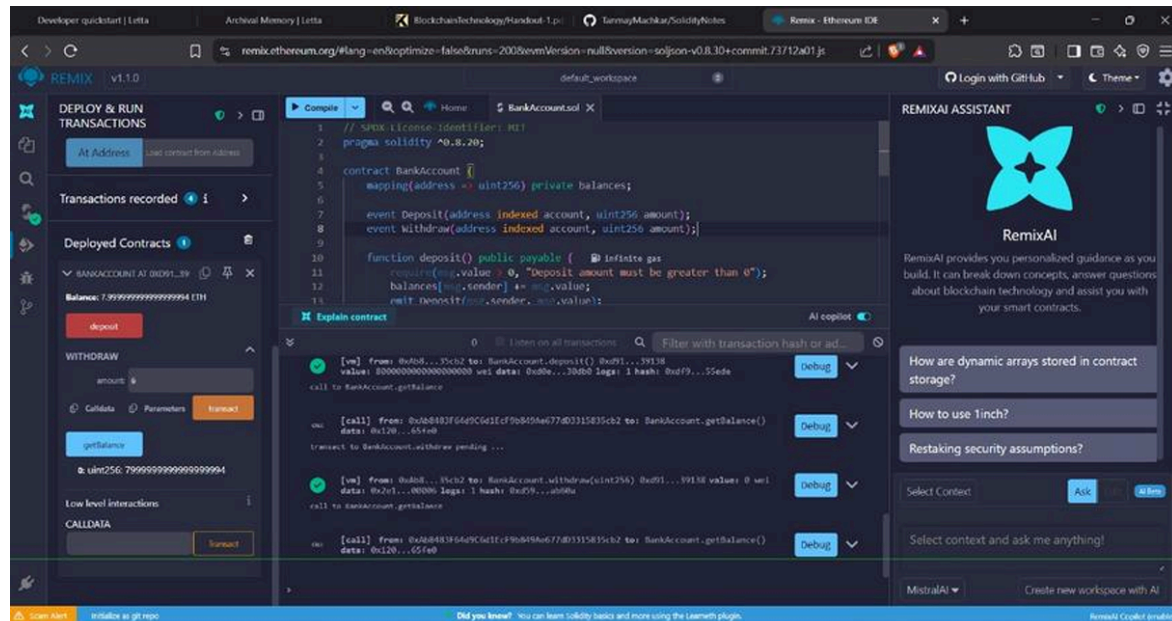
    function withdraw(uint256 amount) public {
        require(amount > 0, "Withdraw amount must be greater than 0");
        require(balances[msg.sender] >= amount, "Insufficient balance");

        balances[msg.sender] -= amount;
        payable(msg.sender).transfer(amount);

        emit Withdraw(msg.sender, amount);
    }

    function getBalance() public view returns (uint256) {
        return balances[msg.sender];
    }
}
```

Output:



Practical 4 (Student Data)

Code:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
```

```
contract StudentData {
    struct Student {
        uint256 id;
        string name;
        uint8 age;
        string course;
    }
}
```

```
Student[] private students;
```

```
event StudentAdded(uint256 id, string name, string course);
event FallbackCalled(address sender, uint value, string message);
```

```
function addStudent(uint256 _id, string memory _name, uint8 _age, string memory
_course) public {
    students.push(Student(_id, _name, _age, _course));
}
```

```

    emit StudentAdded(_id, _name, _course);
}

```

```

function getStudent(uint256 index) public view returns (uint256, string memory, uint8,
string memory) {
    require(index < students.length, "Invalid index");
    Student memory s = students[index];
    return (s.id, s.name, s.age, s.course);
}

```

```

function getTotalStudents() public view returns (uint256) {
    return students.length;
}

```

```

fallback() external payable {
    emit FallbackCalled(msg.sender, msg.value, "Fallback function triggered!");
}

```

```

receive() external payable {
    emit FallbackCalled(msg.sender, msg.value, "Receive function triggered!");
}
}

```

Output:

The screenshot displays the Remix IDE interface with the following components:

- Left Panel (DEPLOY & RUN TRANSACTIONS):**
 - Contract:** 'StudentData.sol' is selected.
 - GETSTUDENT:** The 'index' parameter is set to '1'. The 'call' button is highlighted.
 - Results:**
 - 0: uint256: 1
 - 1: string: tannay
 - 2: uint8: 17
 - 3: string: comp
 - Low level interactions:** Shows a 'CALLDATA' field with the value '13'.
- Center Panel (Code Editor):** Displays the Solidity code for 'StudentData.sol', including the `fallback()` and `receive()` functions.
- Right Panel (Explain contract):** Shows the 'decoded output' and 'raw logs' for the executed transaction. The 'raw logs' section contains the following data:


```

{
  "logIndex": "0x1",
  "blockNumber": "0x1",
  "transactionIndex": "0x1",
  "transactionHash": "0x1",
  "address": "0x1"
}

```